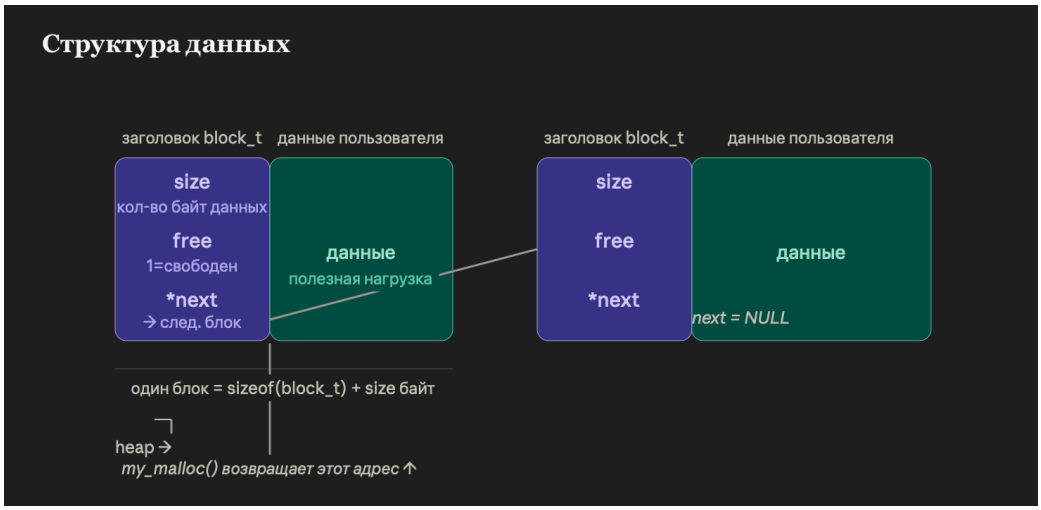
Полный код

```
#include <stdio.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>


// hexdump -C heap.bin

#define HEAP_SIZE (1024)

typedef struct block {
    size_t size;
    int free;
    struct block* next;
} block_t;

block_t* heap = NULL;

void init_heap_file(const char* filename) {
    int fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0666);

    if(fd < 0) { perror("open"); return; }

    if(ftruncate(fd, HEAP_SIZE) != 0) { perror("ftruncate"); return; }

    heap = mmap(NULL, HEAP_SIZE,
                PROT_READ | PROT_WRITE,
                MAP_SHARED,
                fd, 0);

    if(heap == MAP_FAILED) { perror("mmap"); return; }

    heap->size = HEAP_SIZE - sizeof(block_t);
    heap->free = 1;
    heap->next = NULL;

    close(fd);
}

void print_heap() {
    block_t* cur = heap;
    printf("Heap blocks:\n");
    while(cur) {
        printf("  block %p size=%zu free=%d\n", cur, cur->size, cur->free);
        cur = cur->next;
    }
    printf("\n");
}


void wait_for_enter() {
    printf("Press [Enter] to continue...\n");
    while(getchar() != '\n');
}

void* my_malloc(size_t size) {
    block_t* cur = heap;
    while(cur) {
        if(cur->free && cur->size >= size) {
            if(cur->size > size + sizeof(block_t) + 16) {
                block_t* new_block = (block_t*)((char*)cur + sizeof(block_t) +
size);
                new_block->size = cur->size - size - sizeof(block_t);
```

```
                new_block->free = 1;
                new_block->next = cur->next;
                cur->next = new_block;
                cur->size = size;
            }
            cur->free = 0;
            return (char*)cur + sizeof(block_t);
        }
        cur = cur->next;
    }
    return NULL;
}

void my_free(void* ptr) {
    if(!ptr) return;
    block_t* block = (block_t*)((char*)ptr - sizeof(block_t));
    block->free = 1;
}

int main() {
    init_heap_file("heap.bin");

    printf("Initial heap:\n");
    print_heap();
    wait_for_enter();

    void* a = my_malloc(100);
    printf("After malloc 100 bytes (a=%p):\n", a);
    print_heap();
    wait_for_enter();

    void* b = my_malloc(200);
    printf("After malloc 200 bytes (b=%p):\n", b);
    print_heap();
    wait_for_enter();

    my_free(a);
    printf("After free(a):\n");
    print_heap();
    wait_for_enter();

    void* c = my_malloc(50);
    printf("After malloc 50 bytes (c=%p):\n", c);
    print_heap();
    wait_for_enter();

    void* d = my_malloc(50000);
    printf("After malloc 50000 bytes (d=%p):\n", d);
    print_heap();
    wait_for_enter();

    return 0;
}
```

Пояснение

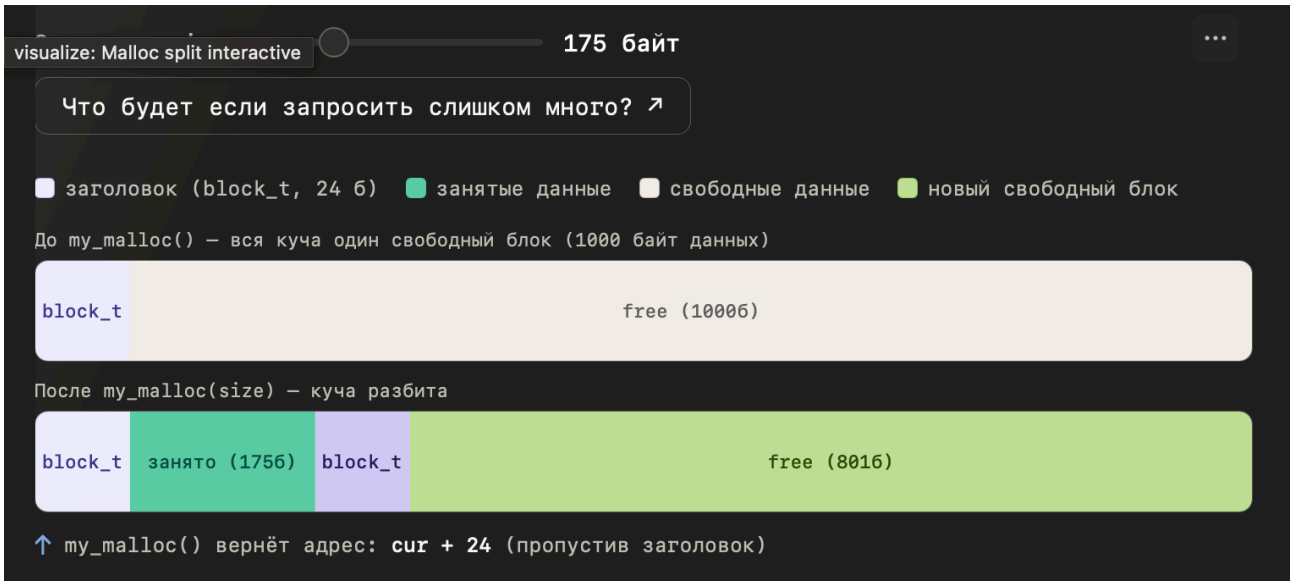
```
#include <stdio.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>


// hexdump -C heap.bin

#define HEAP_SIZE (1024) //Размер типо нашего маллока

typedef struct block { // каждый блок представляет из себя заголовок+ данные
    size_t size; // в заголовке у нас размер блока данных
    int free; // свободен или нет ( если свободен модем использовать - иначе там есть данные )
    struct block* next; //указатель на следующий блок
} block_t;
```




```
block_t* heap = NULL;

void init_heap_file(const char* filename) { // создаем файл с нашим маллоком для наглядности
    int fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0666); // файл с правами чтения и запизи +
trunc - чтобы при каждом запуске программы файл очищался

    if(fd < 0) { perror("open"); return; } //если какая-то ошибка создания файла

    if(ftruncate(fd, HEAP_SIZE) != 0) { perror("ftruncate"); return; } //Устанавливает размер файла
ровно в 1024 байта (резервирует место на диске).

    heap = mmap(NULL, HEAP_SIZE,
                PROT_READ | PROT_WRITE,
                MAP_SHARED,
                fd, 0); mmap отображает файл в адресное пространство процесса — теперь heap это
обычный указатель, но любая запись в него автоматически попадает в файл. map_shared — изменения
видны в файле сразу.

    if(heap == MAP_FAILED) { perror("mmap"); return; }

    heap->size = HEAP_SIZE - sizeof(block_t); // 1024 - 24 = 1000 ( так как у нас есть заголовок и
поэтому используем не всю память) а только 1000 из 1024

    heap->free = 1; // изначально память свободна
    heap->next = NULL; //следующего нет так как у нас весь размер 1024 за не го не выходим
( упрощение в рамках этого кода)

    close(fd);
}

void print_heap() { //функция для удобного вывода чтобы отслужить что происходит с нашей памятью
( последователь итерируемся по блокам чтобы о каждом вывести информамацию)

    block_t* cur = heap; //сначало указываем на начало чтобы по всем блокам потом пройти
    printf("Heap blocks:\n»); //шапочка
    while(cur) { // пока есть следующей ( чтобы пройтись по всем блокам)
        printf("  block %p size=%zu free=%d\n", cur, cur->size, cur->free);  (печатаем все о
конкретном блоке ( его заголовок)
```

```
        cur = cur->next; // перемещаемся на след блок чтобы уже про него написать информацию
    }
    printf(«\n");
}


void wait_for_enter() { // чтобы у нас быстро все не скакало функция что продолжаем выполнения
программы только после нажатия ентер
    printf("Press [Enter] to continue...\n");
    while(getchar() != '\n');
}

void* my_malloc(size_t size) { // непосредственное создание блока памяти
    block_t* cur = heap; // сначало мы должны найти место для блока - начинаем искать с начало нашей
кучки
    while(cur) { // пока есть следующий блок
        if(cur->free && cur->size >= size) { // если блок свободен и и подходит по размеру
            if(cur->size > size + sizeof(block_t) + 16) { // проверяем влезет нужно ли переписывать
заголовок или идеально подходит по размерам


                block_t* new_block = (block_t*)((char*)cur + sizeof(block_t) + size); // считаем
указатель на след блок нужен для заголовка то есть заголовок + размер + где текущий блок начался
                new_block->size = cur->size - size - sizeof(block_t); //размер это следуюзего! Блока

                new_block->free = 1; // помещаем следующий как свободных ( то есть как будто
разрезали ( первый ( который нам нужен он занят второй который остался - свободный
                new_block->next = cur->next; // новый ( отрезанный ссылается на то место на которые
ссылался изначальный ( до разрезания)
                cur->next = new_block; ( текущий (который нужен пользователю ссылается на
отрезанный)
                cur->size = size;  ( размер ну размер)


            }
            cur->free = 0; // просто помещаем что блок занят
            return (char*)cur + sizeof(block_t); // возвращаем указатель на начало место куда можно
писать ( то есть должны отступить служебную информацию от начало блока)
        }
        cur = cur->next; // это для поиска свободного блока - то есть если не подошел по параметрам
переходим по указателю на след блок и уже про if проверяем его
    }
    return NULL;
}
```

Картинка пояснение что было до этого куска и что стало после




```
void my_free(void* ptr) { // функция чтобы пометить что свободен ( просто ставим free =1 )
    if(!ptr) return;
    block_t* block = (block_t*)((char*)ptr - sizeof(block_t));  // укзатель на служебные данные это
просто ссылка на там маллок минус размер служебных - то есть просто арифметика указателей
    block->free = 1;
}
```

```c
int main() {
    init_heap_file("heap.bin");

    printf("Initial heap:\n");
    print_heap();
    wait_for_enter();

    void* a = my_malloc(100);
    printf("After malloc 100 bytes (a=%p):\n", a);
    print_heap();
    wait_for_enter();

    void* b = my_malloc(200);
    printf("After malloc 200 bytes (b=%p):\n", b);
    print_heap();
    wait_for_enter();

    my_free(a);
    printf("After free(a):\n");
    print_heap();
    wait_for_enter();

    void* c = my_malloc(50);
    printf("After malloc 50 bytes (c=%p):\n", c);
    print_heap();
    wait_for_enter();

    void* d = my_malloc(50000);
    printf("After malloc 50000 bytes (d=%p):\n", d);
    print_heap();
    wait_for_enter();

    return 0;
}
```

```
daria@Daria-K:~/labs/OC_labs$ /home/daria/labs/OC_labs/4_part/4.3/4.3
Initial heap:
Heap blocks:
  block 0xfb79667c1000 size=1000 free=1 // изначально просто один большой свободный блок

Press [Enter] to continue...

After malloc 100 bytes (a=0xfb79667c1018): // выделили один блок размера 100
Heap blocks:
  block 0xfb79667c1000 size=100 free=0 // вот он выделенный и занятый
  block 0xfb79667c107c size=876 free=1 // оставщийся свободный ( прошлый занял 100+24)

Press [Enter] to continue...

After malloc 200 bytes (b=0xfb79667c1094): //тут два занятых
Heap blocks:
  block 0xfb79667c1000 size=100 free=0
  block 0xfb79667c107c size=200 free=0
  block 0xfb79667c115c size=652 free=1

Press [Enter] to continue...

After free(a):
Heap blocks:
  block 0xfb79667c1000 size=100 free=1 // вот мы один освободили и теперь он помин как свободный
  block 0xfb79667c107c size=200 free=0
  block 0xfb79667c115c size=652 free=1

Press [Enter] to continue...

After malloc 50 bytes (c=0xfb79667c1018):
Heap blocks:
  block 0xfb79667c1000 size=50 free=0 // выделили 50 то есть разрезали али свободный 100 байт
  block 0xfb79667c104a size=26 free=1
  block 0xfb79667c107c size=200 free=0
  block 0xfb79667c115c size=652 free=1

Press [Enter] to continue...

After malloc 50000 bytes (d=(nil)): //он больше чем нужно не смогли выделить
Heap blocks:
  block 0xfb79667c1000 size=50 free=0
  block 0xfb79667c104a size=26 free=1
```

```
  block 0xfb79667c107c size=200 free=0
  block 0xfb79667c115c size=652 free=1

Press [Enter] to continue...

daria@Daria-K:~/labs/OC_labs$
```